SECTION 2 · SUBSECTION 2.1
SUBAGENTS ON THE DURABLE HARNESS

2.1.1 INTRODUCTION.

Pi-durable has no subagent object. A subagent is an ordinary conversation whose owner is a task instead of nothing. Its lifetime, its stop, its restart and whether its parent is busy all follow from which task owns it. Conduit builds two shapes on that rule: a child owned by the tool call that spawned it, and a child owned by a background anchor task (figure 2.1-1).

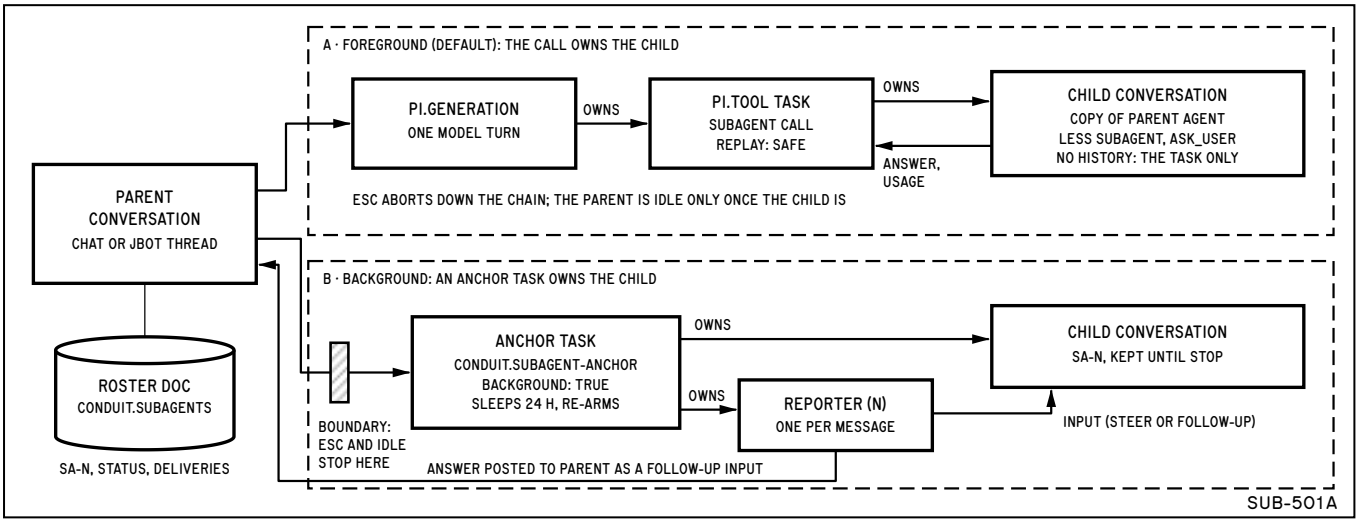


Figure 2.1-1. Subagent Ownership, Foreground and Background

2.1.2 FUNCTIONAL DESCRIPTION.

One "subagent" tool serves both shapes. In the foreground the call creates the child in one commit, strips its SUBAGENT and ASK_USER tools, submits the task and waits; the child's last answer becomes the tool result. With BACKGROUND set, the same commit creates an anchor task owned by the parent conversation, the child under it, and a reporter task that delivers the task. The call returns "Started subagent sa-N" at once. When the child answers, the reporter posts the answer to the parent as a follow-up input, which starts a new parent run (figure 2.1-2).

2.1.2.1 The Ownership Rule.

Owned work belongs to its owner: aborting the owner aborts the child, bottom-up, and the owner is idle only once the child is. A task created with background: true is the boundary at which both stop.

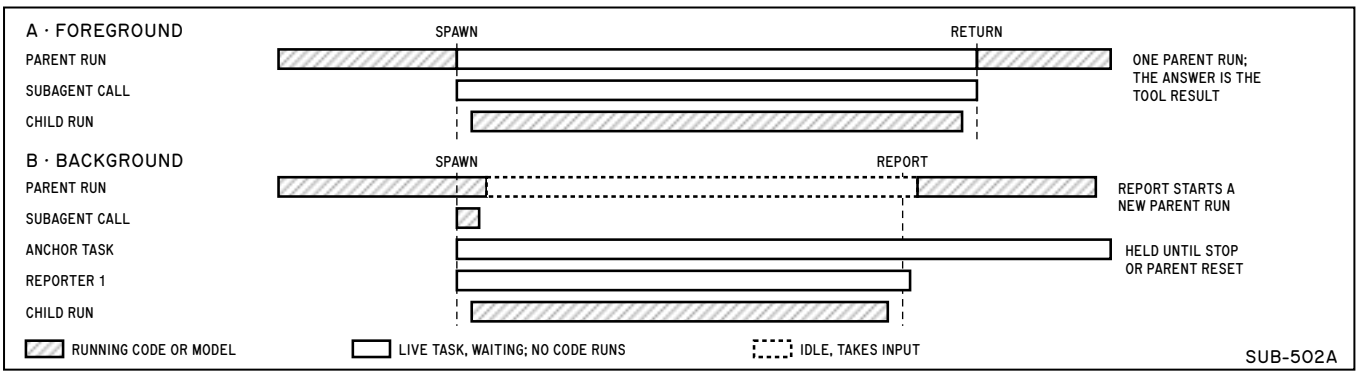


Figure 2.1-2. Lifecycle of a Foreground and a Background Subagent (Not to Scale)

2.1.3 PERFORMANCE AND DESIGN DATA.

Table 2.1-1. Subagent Constants (pi-durable 1.3.0, Conduit)

Parameter	Value
Background id	sa-1, sa-2 … per parent
Tool actions	spawn, message, wait, stop, list
Delegation depth	1 (children lack the tool)
Anchor hold	24 h sleep, re-armed
Foreground activity line	at most 1 per 500 ms
Agents refresh to clients	every 2 s while observed
Foreground restart key	subagent:<task id>
Title	first line of task, 80 chars
Report request id	subagent-report:<child>:<n>

CAUTION

A background child keeps running, and spending, after Esc. Only STOP, a reset of the parent or deleting the chat ends it.

STOP BACKGROUND CHILD	ANY

```
1 RIDER - locate sa-N
2 STOP - press
  √ REPORTERS - aborted first
  √ ANCHOR - aborted last
  √ ROSTER - "stopped"
3 PARENT - verify no report
```

SUB

2.1.4 **RESTART AND REPLAY.**

Every step is a commit, so a restarted hub continues rather than starts again. The foreground call is replay-safe: rerun after a crash, it finds its child through the ownership index and its submission through the request id subagent:<task id>, and waits again. The anchor's sleep is durable. A reporter memoizes the child's answer before posting it, and the report's request id makes a second post a no-op. A spawn inside a codemode script is a nested call keyed by its position; the script's journal replays it to the same child.

2.1.5 **WHAT CONDUIT ADDS.**

Pi-durable supplies ownership, tasks, submissions and the two example patterns. Conduit adds the roster document (ids, status, deliveries, and since this revision the script call that spawned a child), the child's own system prompt (the subagent document, or a run's RUN.md for jbot), an observer that turns the child's events into an activity line, and the report notice. Jbot's routines use the same anchor and reporter without a tool call, through a gated spawn whose report is weighed, counted against the day's budget and sent to the inbox or the home thread.

2.1.6 **PROPOSED PRESENTATION.**

A subagent is not a tool call. The calls are its two ends: the spawn, where it branches off, and the answer, where it joins back. The transcript draws each end where it lands. A foreground child begins and ends inside one turn, so one agent row covers it. A background child branches in one turn and joins in a later one, so it is drawn as a spawn line, a rider while it works, and a report row that links back to the spawn (figure 2.1-3, table 2.1-2).

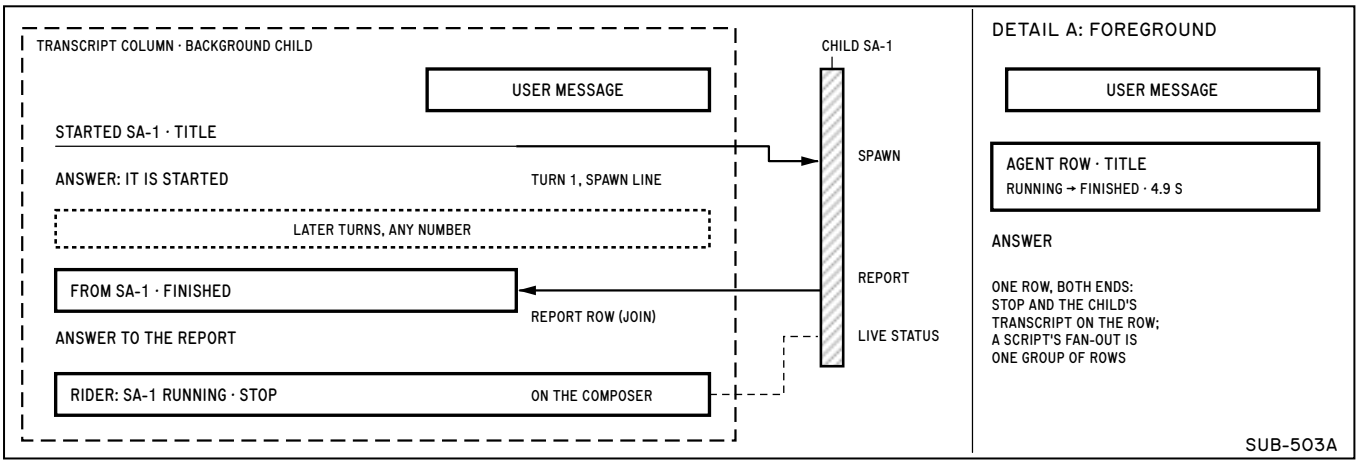


Figure 2.1-3. Proposed Transcript Presentation: Branch and Join

2.1.7 **OPEN DECISIONS.**

2.1.7.1 **Default Shape.**

A foreground child holds the parent's run and dies with Esc; a background child survives Esc but each report costs a full parent run. Which the model reaches for, and for how long a task, is a prompt and tool-description choice.

2.1.7.2 **Report Delivery.**

A report is a follow-up input today, so each one starts a parent run. Pi-durable also accepts a write: an entry with model content is placed at the next boundary and read on the parent's next request without starting a run, so reports could wait, or be batched.

2.1.7.3 **Depth and Inheritance.**

Children cannot delegate. Allowing it deepens the tree; stop and spend follow ownership, but the transcript must draw grandchildren. Every child gets the subagent document as its prompt; its own model and effort are set only for jbot's runs.

2.1.7.4 **Source of Truth.**

The UI reads Conduit's roster. Pi-durable's task graph (owner edges, background flags) and its nested tool events (parentToolCallId) carry the same facts, and other harnesses' subagents could map onto the same two shapes.

Table 2.1-2. Proposed Presentation by Event

Event	Drawn as
Foreground spawn	Agent row in the turn, live
Foreground return	Same row settles; answer in it
Background spawn	Line: "Started sa-N · title"
Background working	Composer rider: status, Stop
Background report	Report row, linked to its spawn
Script fan-out	One group of rows, or one line
Message or steer later	Line: "Messaged sa-N"

NOTE

Foreground spend returns as the call's usage and lands in the parent's ledger. Background spend stays in the child's ledger; the chat's totals add it.

READ A CHILD'S STATE **UI**

```
1 CALL DETAILS - read
  √ conversationId, subagentId
2 ROSTER DOC - read
  √ status, number, via
3 TASK GRAPH - watch
  √ owner edges, background
```

SUB